git

&

GitHub

Control de Versiones Moderno

Desde los fundamentos de Git hasta el flujo colaborativo profesional con GitHub, CI/CD y buenas prácticas.

versioning | colaboración | CI/CD

Despliegue continuo e Integración continua

¿Qué es Git?

Sistema de Control de Versiones Distribuido

Creado por Linus Torvalds en 2005 para el kernel de Linux.

Git guarda ***snapshots*** del estado de tu proyecto en el tiempo — puedes volver a cualquier versión anterior con un solo comando.

`Creado en 2005 · +90% de proyectos lo usan`

Distribuido

Cada desarrollador tiene una copia completa del historial en local.

Historial

Registro inmutable de todos los cambios con autor y fecha.

Ramificación

Crea ramas para trabajar en paralelo sin afectar main.

Las 3 Zonas de Git

Entiende cómo Git rastrea y guarda tus cambios

Working Directory

Zona de trabajo

Aquí editas archivos. Cambios untracked o modified.

```
git status
```

Staging Area

Índice / Stage

Preparas qué cambios formarán parte del próximo commit.

```
git add <archivo>
```

Repository

Historial local

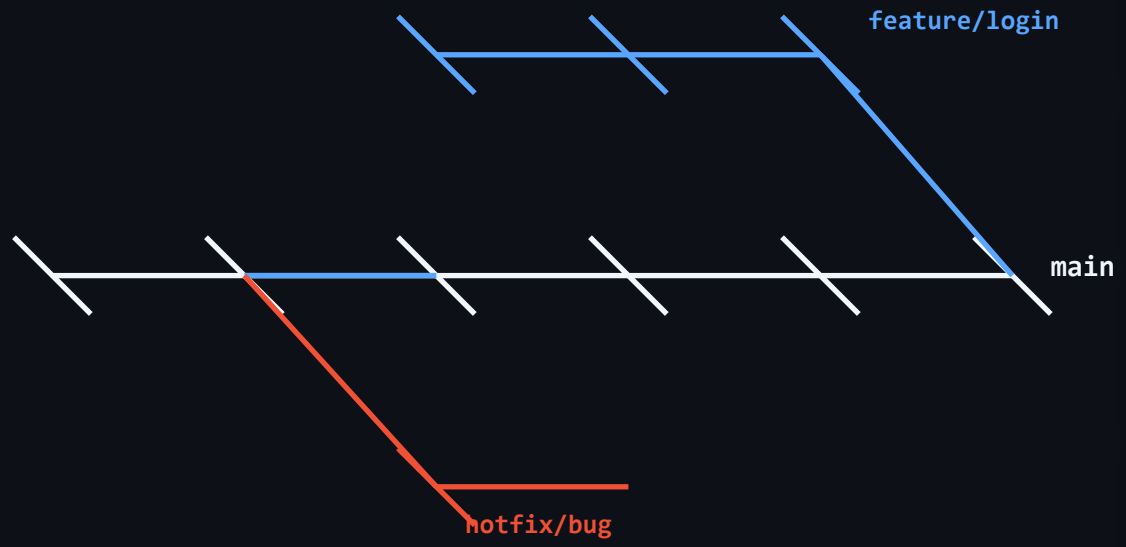
Commits guardados permanentemente en tu historial local.

```
git commit -m '...'
```

Comandos Esenciales

`git init`	Inicializa un nuevo repositorio en el directorio actual
`git clone <url>`	Clona un repositorio remoto completo a tu máquina local
`git add .`	Agrega todos los archivos modificados al staging area
`git commit -m '...'`	Crea un snapshot permanente con un mensaje descriptivo
`git push origin main`	Sube los commits locales al repositorio remoto en GitHub
`git pull`	Descarga e integra los últimos cambios del remoto
`git log --oneline`	Muestra el historial de commits en formato compacto
`git diff`	Muestra los cambios no incluidos en el staging area

Branching & Merging



`git branch feature/login`

Crea una nueva rama

`git checkout feature/login`

Cambia a esa rama

`git checkout -b fix/bug`

Crea y cambia en un paso

`git merge feature/login`

Fusiona la rama en main

`git branch -d feature/login`

Elimina la rama fusionada

¿Qué es GitHub?

GitHub

100M+	Desarrolladores
>420M	Repositorios
4M+	Organizaciones

Repositorios

Almacena y versiona tu código en la nube

Colaboración

Pull Requests y revisión de código entre equipos

GitHub Actions

CI/CD automatizado directamente integrado

Seguridad

Alertas Dependabot y exploración de secretos

GitHub Pages

Publica sitios web desde tu repositorio

GitHub Copilot

IA que escribe código junto a ti

Flujo de Trabajo con GitHub

1

Clone / Fork

```
git clone <url>
```

Copia el repo
a tu máquina

2

Nueva Rama

```
git checkout -b feat
```

Trabaja de forma
aislada

3

Commit

```
git add . && commit
```

Guarda tus
cambios

4

Push

```
git push origin feat
```

Sube la rama
a GitHub

5

Pull Request

```
GitHub → New PR
```

Solicita revisión
del equipo

6

Merge

```
Merge a main
```

Integra los
cambios

GitHub Actions

¿Qué es?

Plataforma de automatización nativa de GitHub. Ejecuta flujos de trabajo (workflows) al detectar eventos en el repositorio, como push o pull_request.

Workflow	Archivo .yml en .github/workflows/
Trigger	on: push, pull_request, schedule...
Job	Conjunto de steps que corren juntos
Step	Comando individual o action reutilizable

Ejemplo de Workflow

```yaml
name: CI Pipeline
on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node
        uses: actions/setup-node@v4
      - name: Install
        run: npm ci
      - name: Test
        run: npm test
```

Buenas Prácticas

Commits atómicos

Un commit = un cambio lógico.
Usa Conventional Commits: feat:, fix:, docs:

Nombres de rama claros

Patrones: feature/login, bugfix/api-timeout, hotfix/prod-crash. Nunca trabajes en main.

Pull Requests revisados

Mínimo 1 aprobación antes de mergear.
Describe qué cambia y por qué.

.gitignore siempre

Excluye node_modules/, .env, *.log, /dist.
Jamás subas credenciales al repositorio.

git pull --rebase

Mantén el historial lineal. Evita merge commits innecesarios al actualizar tu rama.

README descriptivo

Documenta qué hace el proyecto, cómo instalarlo y cómo contribuir al mismo.

¡Empieza a versionar hoy!

Git y GitHub son habilidades fundamentales del desarrollador moderno.
La práctica diaria construye el hábito.

git-scm.com

Documentación oficial de Git

docs.github.com

Guías y referencia de GitHub

learngitbranching.js

Aprende branching visualmente

```
$ git commit -m 'inicio de una nueva etapa profesional'
```